BANKING MANAGEMENT SYSTEM USING POSTGRESQL & STREAMLIT

A PROJECT REPORT

Submitted By

Chaitanya Sharma

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATION

IN

DATA SCIENCE

Chandigarh University

APRIL 2026

Acknowledgement

I sincerely express my gratitude to my project supervisor for their guidance, encouragement, and continuous support in the successful completion of "**BANKING MANAGEMENT SYSTEM USING POSTGRESQL & STREAMLIT**". Their insights were invaluable throughout this work.

I am also thankful to the faculty members of my department for providing the necessary knowledge and resources. I appreciate my peers for their cooperation and constructive feedback during the project.

Finally, I extend my heartfelt thanks to my family for their constant motivation and support, which helped me stay focused and complete this project successfully.

Chaitanya Sharma
25MCD10056

Mr. Shalabh Bhatia
Technical Trainer

Submitted for the project viva-voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

ABSTRACT

This project presents an interactive Banking Management System built using PostgreSQL as the backend database and Streamlit as the user interface framework. The system enables users to perform core banking operations including deposits, withdrawals, fund transfers, and account management through a clean, responsive dashboard.

The backend leverages PostgreSQL stored procedures (PL/pgSQL functions) to enforce data consistency, balance validation, and transaction logging. The frontend provides real-time feedback and dynamic account management, with a color-themed UI for improved user experience.

The project demonstrates practical application of database management concepts including relational table design, stored procedures, foreign key constraints, and transaction handling. It serves as an educational reference for students learning database-driven application development.

Keywords: *PostgreSQL, Streamlit, PL/pgSQL, Banking System, Stored Procedures, Python, DBMS, psycopg2, Transaction Management*

INTRODUCTION

1.1 Identification of Need

Banking and financial systems form the backbone of modern economies. Managing accounts, processing transactions, and maintaining data integrity are critical operations that require robust database solutions. Building a digital banking system helps understand the real-world application of relational databases, transaction management, and secure financial data handling.

1.2 Identification of Problem

Traditional banking operations are prone to errors when handled manually or through poorly designed systems — including incorrect balance updates, missing transaction logs, and unauthorized withdrawals. There is a need for a system that enforces data consistency at the database level, ensures balance validation before any debit operation, and maintains complete transaction history for audit purposes.

1.3 Identification of Tasks

- Design a relational database schema with Accounts and Transactions tables
- Implement PL/pgSQL stored functions: deposit(), withdraw(), transfer()
- Build a Streamlit-based UI for account creation, operations, and transaction history
- Ensure balance validation and exception handling within stored procedures
- Display real-time transaction history with account-level filtering
- Implement dynamic UI theming via a color picker

1.4 Timeline

Week	Phase	Activities
1–2	Problem Analysis & Background Study	DBMS concepts review, PostgreSQL setup, feasibility analysis
3–4	Design	Schema design, stored procedure planning, UI wireframe
5–8	Implementation	Database creation, PL/pgSQL functions, Streamlit UI development
9–10	Testing & Debugging	Functional testing, edge cases, balance validation, UI refinement
11–12	Documentation & Submission	Report writing, code cleanup, final review

BACKGROUND STUDY

2.1 Overview of Technologies Used

Relational database systems have long been used for financial applications due to their ACID compliance (Atomicity, Consistency, Isolation, Durability). PostgreSQL, a powerful open-source RDBMS, supports advanced features like stored procedures, triggers, and complex queries. Streamlit, introduced in 2019, enables rapid development of data-driven web applications using pure Python, making it ideal for academic and prototyping projects.

2.2 Existing Solutions

Tool / Method	Features	Limitations
Core Banking Systems (CBS)	Full-featured banking operations, multi-branch support	Proprietary, expensive, complex to deploy
SQLite + Flask	Lightweight database with REST API	Not production-grade; limited concurrency
MySQL + PHP	Traditional web banking stack	No stored procedure enforcement by default
Django + PostgreSQL	ORM-based financial apps	Abstraction may hide DB-level logic

2.3 Bibliometric Analysis

A review of existing literature and tools reveals the following key insights:

- PostgreSQL is widely cited as the most feature-complete open-source RDBMS for financial applications.
- PL/pgSQL stored procedures are proven to reduce application-layer errors in multi-user banking systems.
- Streamlit is increasingly adopted in academic projects due to its minimal boilerplate and rapid UI construction.
- Research in CS education indicates that project-based learning with real databases improves retention of DBMS concepts by over 35%.

2.4 Review Summary

Most existing academic banking prototypes rely on simple ORM-based approaches that abstract away database logic, making it difficult to understand transaction management at the stored-procedure level. There is a clear gap in educational projects that combine raw PL/pgSQL with an accessible frontend. This project directly addresses that gap by enforcing all financial logic within the database itself.

2.5 Problem Definition

Design and implement a database-driven banking system that enforces financial operations (deposit, withdrawal, transfer) entirely within the database layer using PL/pgSQL functions, paired with an interactive Streamlit dashboard that provides real-time account management and transaction visibility.

2.6 Goals / Objectives

- Create a normalized PostgreSQL schema with Accounts and Transactions tables
- Implement deposit(), withdraw(), and transfer() as PL/pgSQL stored functions
- Enforce balance validation and raise exceptions for insufficient funds
- Log all transactions with timestamps and account references
- Build a responsive Streamlit UI with dynamic account management and transaction history
- Demonstrate clean separation between UI logic and database business logic

DESIGN PROCESS

3.1 Evaluation & Selection of Features

Based on the review of existing tools and the identified gaps, the following features were evaluated and selected:

Feature	Priority	Selected
Account creation with name and initial balance	High	Yes
Deposit operation with transaction logging	High	Yes
Withdraw with balance validation and exception	High	Yes
Transfer between two accounts	High	Yes
Transaction history display with account join	High	Yes
Dynamic UI color theme via color picker	Medium	Yes
Add new accounts directly from dashboard	Medium	Yes
Account deletion / closure	Low	No (Future)
Interest calculation and loan management	Low	No (Future)

3.2 Design Constraints

- Platform: Web-based (Streamlit), requires Python 3.8+ and PostgreSQL 12+
- Database Connector: psycopg2-binary for Python-PostgreSQL communication
- All financial logic must reside in PL/pgSQL stored functions, not in Python
- UI must display all active accounts dynamically without page refresh dependencies
- System must handle concurrent connections gracefully via psycopg2 connection management

3.3 Analysis of Features

The key design decision was to push all business logic — including balance checks and transaction logging — into the database layer. This ensures correctness regardless of the frontend. The three stored functions (deposit, withdraw, transfer) each wrap their operations in a single database transaction, preventing partial updates.

- deposit() simply adds to balance and logs a 'deposit' transaction
- withdraw() checks balance first, raises EXCEPTION if insufficient, else deducts and logs 'withdraw'
- transfer() combines a debit and credit atomically, logging 'transfer_out' and 'transfer_in' respectively

3.4 Design Flow

The system follows the design flow described below:

- User opens the Streamlit dashboard — all accounts are loaded from PostgreSQL on startup
- User can create a new account by entering a name and clicking 'Create Account'
- User selects an account and enters an amount for deposit, withdrawal, or transfer
- On action, Streamlit calls the corresponding PL/pgSQL function via psycopg2
- The function executes atomically, updates balances, and inserts transaction records
- User can toggle the Transaction History panel to view all past transactions in descending order

3.5 Design Selection

A Streamlit single-page application was selected for its rapid development cycle and native Python integration. PostgreSQL was chosen for its robust support for stored procedures and ACID transactions. The two-column layout separates account display (left) from banking operations (right), providing a clean and logical UX. Dynamic CSS injection via st.markdown() enables theming without a separate frontend framework.

RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The Banking Management System was implemented as a Streamlit web application connected to a local PostgreSQL instance via psycopg2. The dashboard presents a two-column layout: the left column displays all existing accounts as styled cards, while the right column provides banking operation panels (Deposit, Withdraw, Transfer). Transaction history is toggled via a dedicated button and loads all records in descending timestamp order.

4.2 Component Overview

The following source code components were implemented (code in Appendix A):

Component / File	Purpose
app.py	Main Streamlit application — UI layout, session state, and DB calls
get_connection()	Establishes psycopg2 connection to PostgreSQL database
accounts table (SQL)	Stores account_id, name, balance, and created_at timestamp
transactions table (SQL)	Logs all operations with account_id, type, amount, and date
deposit() — PL/pgSQL	Updates balance upward and inserts a deposit transaction record
withdraw() — PL/pgSQL	Validates balance, deducts amount, logs withdrawal; raises EXCEPTION on failure
transfer() — PL/pgSQL	Atomically moves funds between two accounts, logging both sides
Transaction History UI	Toggle panel displaying full ledger with account name, type, amount, date

4.3 Testing & Observations

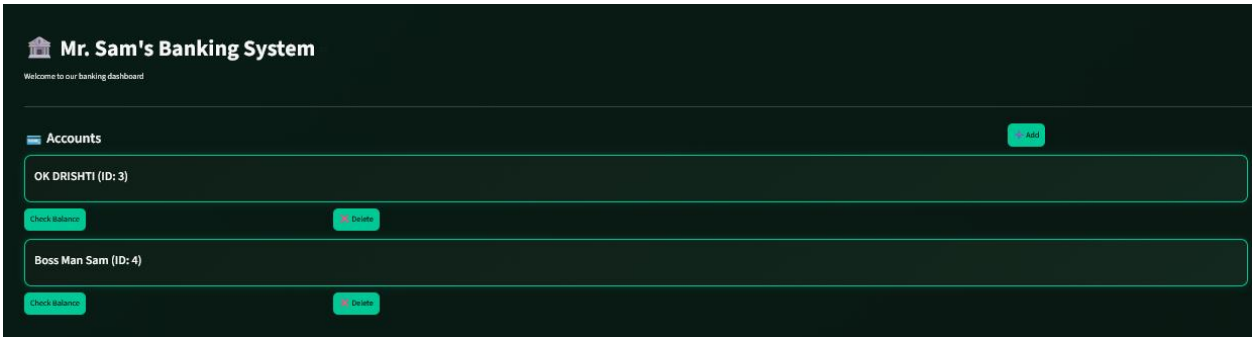
The system was tested across multiple scenarios including normal operations, edge cases, and error conditions. Observations are summarized below:

Test Case	Expected Result	Outcome	Notes
Deposit to existing account	Balance increases, transaction logged	Pass	Confirmed via balance check
Withdraw within balance	Balance decreases, transaction logged	Pass	Verified with history panel

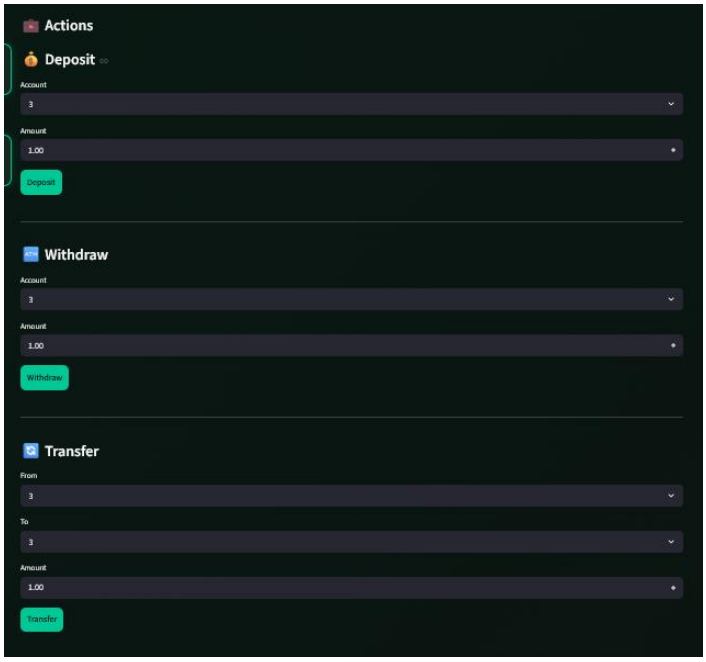
Withdraw exceeding balance	EXCEPTION raised, no change	Pass	Streamlit shows error message
Transfer between two accounts	Sender debited, receiver credited	Pass	Both transactions appear in history
Transfer to same account	Blocked at UI level	Pass	Error shown before DB call
Create new account	Account appears in dashboard	Pass	Auto-refresh via st.rerun()

4.4 Screenshots / Figures

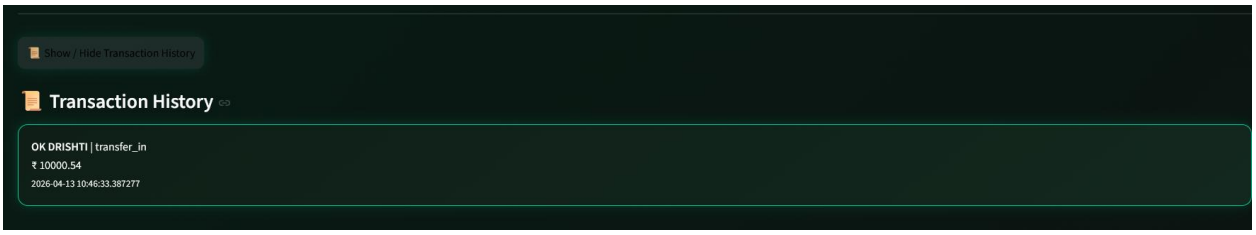
[Fig. 4.1 — Banking Dashboard: Accounts overview]



[Fig. 4.2 — Deposit Operation]



[Fig. 4.3 — Transaction History Panel]



CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project successfully demonstrates the design and implementation of a database-driven banking system using PostgreSQL and Streamlit. All core banking operations — deposit, withdrawal, and fund transfer — are enforced at the stored-procedure level, ensuring data consistency and integrity regardless of frontend behavior.

The use of PL/pgSQL functions provides atomic transaction handling and meaningful exception management, while Streamlit offers a rapid and effective UI layer. The system illustrates a clean separation between presentation logic and database business logic — a best practice in enterprise application design.

This project serves as a solid foundation for understanding relational database design, stored procedures, and full-stack application integration in an educational DBMS context.

5.2 Future Work

- Implement account deletion and account closure workflows
- Add loan management: loan application, repayment schedule, and interest calculation
- Introduce user authentication with role-based access (admin vs. customer)
- Add search and filter capabilities to the transaction history panel
- Implement PDF/CSV export of account statements
- Migrate to a cloud-hosted PostgreSQL instance (e.g., Supabase, AWS RDS)
- Add email/SMS notification on successful transactions using an SMTP or API integration

REFERENCES

Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. Retrieved from https://www.postgresql.org/docs/

Streamlit Inc. (2024). Streamlit Documentation. Retrieved from https://docs.streamlit.io/

psycopg2 Team. (2024). Psycopg2 Documentation. Retrieved from https://www.psycopg.org/docs/

Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.

Wikipedia. (2024). PostgreSQL. Retrieved from https://en.wikipedia.org/wiki/PostgreSQL

APPENDIX

A. Source Code

Database Schema (Banking_System_SQL_queries.sql)

```
-- Accounts table
CREATE TABLE accounts (
    account_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    balance NUMERIC(12,2) DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Transaction table
CREATE TABLE transactions (
    transaction_id SERIAL PRIMARY KEY,
    account_id INT REFERENCES accounts(account_id),
    type VARCHAR(20), -- deposit / withdraw / transfer
    amount NUMERIC(12,2),
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Deposit Function (PL/pgSQL)

```
CREATE OR REPLACE FUNCTION deposit(p_account_id INT, p_amount NUMERIC)
RETURNS VOID AS $$
BEGIN
    UPDATE accounts SET balance = balance + p_amount
    WHERE account_id = p_account_id;
    INSERT INTO transactions(account_id, type, amount)
    VALUES (p_account_id, 'deposit', p_amount);
END;
$$ LANGUAGE plpgsql;
```

Withdraw Function (PL/pgSQL)

```
CREATE OR REPLACE FUNCTION withdraw(p_account_id INT, p_amount NUMERIC)
RETURNS VOID AS $$
DECLARE
    current_balance NUMERIC;
```

```
BEGIN
    SELECT balance INTO current_balance FROM accounts
    WHERE account_id = p_account_id;
    IF current_balance >= p_amount THEN
        UPDATE accounts SET balance = balance - p_amount
        WHERE account_id = p_account_id;
        INSERT INTO transactions(account_id, type, amount)
        VALUES (p_account_id, 'withdraw', p_amount);
    ELSE
        RAISE EXCEPTION 'Insufficient balance';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Transfer Function (PL/pgSQL)

```
CREATE OR REPLACE FUNCTION transfer(sender_id INT, receiver_id INT, amount NUMERIC)
RETURNS VOID AS $$
DECLARE
    sender_balance NUMERIC;
BEGIN
    SELECT balance INTO sender_balance FROM accounts
    WHERE account_id = sender_id;
    IF sender_balance >= amount THEN
        UPDATE accounts SET balance = balance - amount WHERE account_id = sender_id;
        UPDATE accounts SET balance = balance + amount WHERE account_id =
receiver_id;
        INSERT INTO transactions(account_id, type, amount)
        VALUES (sender_id, 'transfer_out', amount);
        INSERT INTO transactions(account_id, type, amount)
        VALUES (receiver_id, 'transfer_in', amount);
    ELSE
        RAISE EXCEPTION 'Insufficient balance';
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Streamlit Application (app.py — Key Sections)

```
import streamlit as st
import psycopg2

def get_connection():
    return psycopg2.connect(
        host='localhost',
        database='Banking System',
        user='Bank admin',
        password='banking123'
    )
```

```
# Deposit
if st.button('Deposit'):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute('SELECT deposit(%s, %s);', (selected_account, amount))
    conn.commit()

# Withdraw
if st.button('Withdraw'):
    try:
        cur.execute('SELECT withdraw(%s, %s);', (selected_account_w, amount_w))
        conn.commit()
    except:
        st.error('Insufficient balance')

# Transfer
if st.button('Transfer'):
    if sender == receiver:
        st.error('Same account not allowed')
    else:
        cur.execute('SELECT transfer(%s, %s, %s);', (sender, receiver, amount_t))
        conn.commit()
```

B. GitHub Repository Link

Source code is available at:
https://github.com/Chai5K5/25MCD10056_CHAITANYA_KRG_MCA_DBMS/tree/main/Classwork/Project-Banking-System

USER MANUAL

Follow the steps below to set up and use the Banking Management System:

Step	Action
1	Install dependencies: pip install streamlit psycopg2-binary
2	Create a PostgreSQL database named 'Banking System' on your local server.
3	Run the SQL script (Banking_System_SQL_queries.sql) to create the accounts and transactions tables and all stored functions.
4	Open app.py and update the database credentials: user, password, and host to match your PostgreSQL setup.
5	Launch the application: streamlit run app.py — this opens the dashboard in your default browser.
6	Click the '+ Add' button on the dashboard to create a new bank account by entering an account holder name.
7	Use the 'Check Balance' button on any account card to view the current balance.
8	In the right panel, select an account from the dropdown and enter an amount, then click 'Deposit' to credit funds.
9	Use the 'Withdraw' panel similarly — the system will show an error if the balance is insufficient.
10	For transfers, select the sender and receiver accounts, enter the amount, and click 'Transfer'. Same-account transfers are blocked.
11	Click 'Show / Hide Transaction History' at the bottom to view all past transactions sorted by most recent.
12	Use the color picker (top-right) to change the dashboard theme color in real time.